COESIONE ITALIA 21-27 PIEMONTE

Cofinanziato dall'Unione europea

REGIONE PIEMONTE

DOCENTE Seyedhossein Javadizavieh

UNITA' FORMATIVA Amazon Web Services - Introduzione e utilizzo

ARGOMENTO Lezione 06 - Immagini, container e registri

intervento realizzato da

ITS ACADEMY ICT·PIEMONTE

Agenda

1. Concetto di container
2. Immagini
3. Registri
4. Ciclo di utilizzo
5. Mini-exercises
6. Lab + cleanup
7. Wrap-up + suggested links

Obiettivi

- Distinguere immagine e container.
- Descrivere il ruolo di un registro.
- Rappresentare il ciclo di utilizzo di un'immagine.

«Sul mio PC funziona»: quale artefatto consegnare?

Vogliamo eseguire lo stesso server web due volte, distinguendo versione del software e singole esecuzioni.

Un container è un insieme di processi isolati; non è una piccola macchina virtuale con un proprio kernel. I container Linux condividono il kernel dell'host Linux; Docker Desktop può fornire quell'host tramite una VM.

L'immagine impacchetta software e filesystem iniziale, non ogni proprietà della macchina che lo eseguirà.

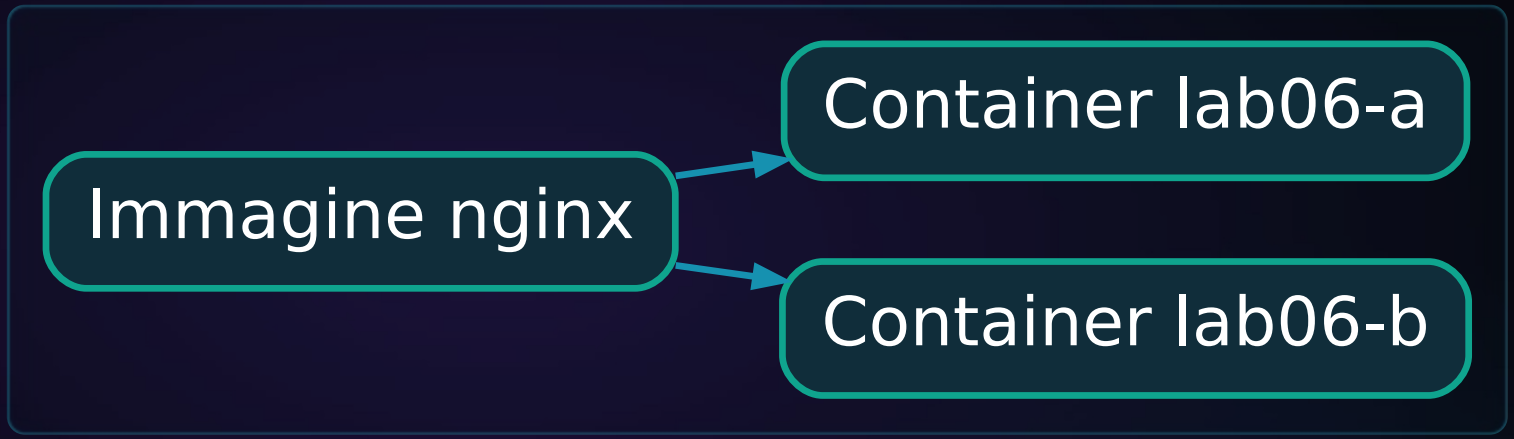
Prima acquisire l'immagine, poi eseguirla

```
docker version
docker pull nginx:alpine
docker image inspect nginx:alpine --format '{{json .RepoDigests}}'
```

`docker version` deve mostrare anche il server Docker. Il pull trasferisce l'artefatto: **non avvia il web server**.

`alpine` è un tag mobile: registrare il digest ottenuto. Non presentare lo stesso tag come garanzia di identico contenuto in date diverse.

Un artefatto, due esecuzioni indipendenti




```
docker run -d --name lab06-a -p 127.0.0.1:8081:80 nginx:alpine
docker run -d --name lab06-b -p 127.0.0.1:8082:80 nginx:alpine
```

Aprire `http://127.0.0.1:8081` e `:8082`. Le porte host differiscono; entrambi i processi ascoltano sulla porta 80 del proprio container.

Cosa cambia quando cambia un container?

Parte	Proprietà	Conseguenza
Layer dell'immagine	Riutilizzati, in sola lettura	Due esecuzioni condividono la base
Layer scrivibile	Proprio del container	Non modifica l'immagine originale
Volume separato	Ciclo di vita distinto	Può sopravvivere al container

Installare manualmente qualcosa in un container in esecuzione non rende ripetibile la distribuzione: nella prossima lezione dichiareremo la costruzione nel Dockerfile.

Il registry distribuisce, non esegue l'applicazione



Push e pull trasferiscono artefatti; `run` crea l'esecuzione. Un registry può essere pubblico o privato: accesso e provenienza vanno verificati separatamente.

Leggere un riferimento completo

```
localhost:5000/lab06-nginx:1.0
registry       repository  tag
```

Il tag è un riferimento modificabile, salvo regole del registry. Il digest identifica il contenuto del manifest; per immagini multi-piattaforma distinguere indice e manifest della piattaforma.

L'ID immagine locale e il digest mostrato dal registry non sono necessariamente la stessa stringa: confrontare identificatori dello stesso tipo.

Pubblicare in un registry didattico locale

```
docker volume create lab06-registry-data
docker run -d --name lab06-registry -p 127.0.0.1:5000:5000 --mount source=lab06-
registry-data,target=/var/lib/registry registry:2
docker tag nginx:alpine localhost:5000/lab06-nginx:1.0
docker push localhost:5000/lab06-nginx:1.0
```

Questo registry senza TLS/autenticazione è limitato al computer del lab. Non pubblicarlo in rete e non riutilizzarlo come configurazione di produzione.

Dimostrare il pull, non soltanto scriverlo

```
docker image rm localhost:5000/lab06-nginx:1.0
docker pull localhost:5000/lab06-nginx:1.0
docker image inspect localhost:5000/lab06-nginx:1.0 --format '{{.Id}}'
docker image inspect nginx:alpine --format '{{.Id}}'
```

Gli ID locali devono coincidere in questo esperimento. Rimuovere il tag non elimina necessariamente i layer: Docker può riusarli dalla cache durante il pull.

Il digest non è un certificato di sicurezza

Il digest rende identificabile un contenuto. Non dimostra che quel contenuto sia privo di vulnerabilità, provenga dal fornitore atteso o non contenga credenziali.

Controlli distinti: fonte e provenienza, dipendenze, vulnerabilità, permessi di esecuzione e segreti esclusi dall'artefatto.

Nel Learner Lab non inserire mai access key, session token o file `.aws` in immagini e contesti di build.

Fermato non significa eliminato

```
docker stop lab06-a
docker ps
docker ps -a --filter name=lab06-a
docker logs --tail 10 lab06-a
```

Il container fermato compare con `ps -a`; conserva configurazione e layer scrivibile. Il container B continua a funzionare. Solo `rm` elimina il container; il volume del registry richiede un cleanup separato.

Se il comando fallisce, leggere il confine

Sintomo	Primo controllo
Cannot connect to Docker daemon	Docker Desktop/Engine avviato
Port is already allocated	Porta host occupata; non terminare processi altrui
Pull access denied	Nome del repository e autorizzazione
No matching manifest	Piattaforma CPU/OS supportata

Il messaggio d'errore è un'evidenza: conservarlo senza copiare credenziali.

Cleanup circoscritto alle risorse del caso

```
docker rm -f lab06-a lab06-b lab06-registry
docker volume rm lab06-registry-data
docker image rm localhost:5000/lab06-nginx:1.0
```

Eseguire solo sui nomi creati durante questo lab. Non usare `system prune`: potrebbe coinvolgere altri progetti. Conservare le immagini condivise se già presenti prima dell'esercizio.

Mini-esercizio Sessione 06 - Immagine e container

Qual è la relazione corretta tra immagine e container?

- A) Sono sempre lo stesso oggetto in esecuzione
- B) L’immagine è il modello; il container è un’esecuzione
- C) Il container è il registro delle immagini



Mini-esercizio Sessione 06 - Immagine e container (risposte)

Qual è la relazione corretta tra immagine e container?

- B) L’immagine è il modello; il container è un’esecuzione

Risposte attese:

- **Corretta: B** - Una stessa immagine può avviare più container.
- A è errata: Immagine e processo in esecuzione sono concetti distinti.
- C è errata: Il registro conserva immagini, non è un container.

Mini-esercizio Sessione 06 - Tag di versione

Perché si registra il tag dell’immagine nella configurazione di rilascio?

- A) Per cambiare il colore del terminale
- B) Per identificare e riprodurre la versione usata
- C) Per evitare ogni controllo di sicurezza



Mini-esercizio Sessione 06 - Tag di versione (risposte)

Perché si registra il tag dell’immagine nella configurazione di rilascio?

- B) Per identificare e riprodurre la versione usata

Risposte attese:

- **Corretta: B** - Il tag collega il rilascio a una versione identificabile.
- A è errata: L’aspetto del terminale è irrilevante.
- C è errata: Il tag non sostituisce i controlli di sicurezza.

Mini-esercizio Sessione 06 - Contenuto minimo

Perché non si devono inserire segreti e file inutili in un’immagine?

- A) Rendono sempre più veloce il pull
- B) Aumentano rischio, dimensione e superficie di attacco
- C) Sono necessari per ogni container



Mini-esercizio Sessione 06 - Contenuto minimo (risposte)

Perché non si devono inserire segreti e file inutili in un’immagine?

- B) Aumentano rischio, dimensione e superficie di attacco

Risposte attese:

- **Corretta: B** - Un’immagine minima e senza segreti riduce esposizione e complessità.
- A è errata: Più contenuto tende ad aumentare il trasferimento.
- C è errata: I file inutili non sono necessari all’esecuzione.

Mini-esercizio Sessione 06 - Cleanup degli artefatti

Un container di prova è fermo e l’immagine non serve più. Che cosa si fa?

- A) Si conservano sempre senza controllo
- B) Si rimuovono entrambi dopo la verifica
- C) Si cambia soltanto il tag



Mini-esercizio Sessione 06 - Cleanup degli artefatti (risposte)

Un container di prova è fermo e l’immagine non serve più. Che cosa si fa?

- B) Si rimuovono entrambi dopo la verifica

Risposte attese:

- **Corretta: B** - Il cleanup evita accumulo di container e immagini inutilizzati.
- A è errata: La conservazione indiscriminata crea confusione.
- C è errata: Cambiare tag non rimuove gli artefatti.

Lab/Exercises

- Esaminare il rapporto tra immagine e container.
- Simulare pubblicazione e recupero da un registro.
- Applicare nomi e versioni coerenti agli artefatti.

Cleanup: fermare e rimuovere i container di prova, poi rimuovere immagini e artefatti temporanei.

Link: Immagini, container e registri

Wrap-up Lesson 06

- L'immagine e un artefatto; il container e la sua esecuzione.
- Il registro conserva e distribuisce immagini.
- Nomi, versioni e cleanup sostengono la riproducibilita.

Lesson 06 - Suggested links

- https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
- https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/
- https://docs.docker.com/docker-hub/repos/